

A HANDS-ON, CODE-FIRST COURSE

LangChain From Zero

Build real GenAI apps with Gemini & local models

PHASE 10 — LANGGRAPH BASICS

Prerequisites: Phases 5 & 9 done — you've used `StateGraph`, `MessagesState`, and checkpoints via memory and agents.

Targets: State & reducers · nodes · edges · conditional branching · loops · streaming graph steps

Estimated time for this phase: 90–130 minutes

Where this phase sits

You've *used* LangGraph indirectly — memory (Phase 5) and agents (Phase 9) are built on it. Now you open the engine and build stateful workflows **by hand**: nodes that do work, edges that route, branches that decide, and **loops** — the one thing LCEL chains can't do.

Ph 0–4 ✓ **Setup** → **Fundamentals** → **Models** → **Prompts** → **Chains**

Ph 5 ✓ **Memory** — a one-node graph + checkpointer.

Ph 6–8 ✓ **Documents** · **Embeddings** · **RAG**

Ph 9 ✓ **Tools & Agents** — `create_agent` (a graph under the hood).

Ph 10 **LangGraph Basics** ← you are here
State & reducers, nodes, edges, conditional routing, loops, streaming steps.

Ph 11 **Observability & Deployment**

Ph 12 **Capstone Project**

Phase 10 · LangGraph Basics

WHAT YOU'LL BE ABLE TO DO AFTER THIS PHASE

Explain when a graph beats a chain or a prebuilt agent; design a typed **State** with reducers; write **nodes** and connect them with **edges**; branch with **conditional edges**; build a real **loop** that repeats until a condition is met; and stream/inspect a graph's execution — all on both Gemini and Ollama.

In this phase

1. 10.1 — Why LangGraph? Loops, branches & shared state
2. 10.2 — The three primitives: State, Nodes, Edges
3. 10.3 — State & reducers (overwrite vs append)
4. 10.4 — A first graph: two nodes in sequence
5. 10.5 — Conditional edges: branching
6. 10.6 — Loops: the thing chains can't do
7. 10.7 — Streaming & inspecting a graph
8. 10.8 — When to use a chain vs an agent vs a graph
9. 10.9 — Common errors & troubleshooting
10. 10.10 — Mini practice exercise
11. Terms introduced this phase

10.1 — Why LangGraph? Loops, branches & shared state

LCEL chains (Phases 1–8) are **linear and acyclic** — data flows one way, start to finish, no going back. That's perfect for fixed pipelines. But many real workflows need to *loop* (try again until good), *branch* (different paths for different inputs), and share a *growing state* across steps. That's LangGraph.

TERM: LANGGRAPH

LangGraph is a library (built on LangChain) for workflows shaped as a **graph**: *nodes* (steps that do work) connected by *edges* (routes between steps), all sharing one *state* object. Unlike chains, graphs can contain **cycles** (loops) and **branches** (decisions). It's the engine under `create_agent` — now you'll drive it directly.

YOU'VE ALREADY SEEN IT TWICE

Phase 5's chatbot was a one-node graph with a checkpointer. Phase 9's agent was a graph with a model node and a tool node looping between them. This phase is just learning to build such graphs yourself — for when a prebuilt agent isn't the exact shape you need.

10.2 — The three primitives: State, Nodes, Edges

Primitive	What it is	In code
State	A shared data object passed to every node — the graph's "memory" during a run.	A <code>TypedDict</code> of fields.
Node	A step that does work: reads the state, returns a partial update to it.	A function <code>state → {updates}</code> .
Edge	A route from one node to the next. Can be fixed, or conditional (decided at runtime).	<code>add_edge</code> / <code>add_conditional_edges</code> .

Two special markers, `START` and `END`, mark where the graph begins and finishes.

10.3 — State & reducers (overwrite vs append)

The State is a `TypedDict` listing the fields the graph carries. By default, when a node returns a field, it **overwrites** the old value. But for fields that should *accumulate* (like a message list), you attach a **reducer** that says "append instead."

```
from typing import Annotated, TypedDict
from operator import add
from langgraph.graph.message import add_messages

class State(TypedDict):
    topic: str # default reducer: OVERWRITE
    drafts: Annotated[list, add] # reducer "add": APPEND to the list
    messages: Annotated[list, add_messages] # message-aware append (Phase 5)
```

TERM: REDUCER

A **reducer** decides how a node's returned value *merges* into the existing state for that field. No reducer ⇒ overwrite (the new value replaces the old). `Annotated[list, add]` ⇒ concatenate lists. `add_messages` ⇒ a smart append for message lists (it also handles updates by id). Pick overwrite for "current value" fields (a counter, a flag) and append for "history" fields.

10.4 — A first graph: two nodes in sequence

Let's build a tiny content pipeline: node 1 writes an outline, node 2 expands it into a post. State carries the topic plus both outputs.

```
START → [outline] → [write] → END
      writes      expands outline
      outline    into a post
```

```

from dotenv import load_dotenv
from typing import TypedDict
from langgraph.graph import StateGraph, START, END          # 1
from langchain_google_genai import ChatGoogleGenerativeAI

load_dotenv()
model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")

class State(TypedDict):                                     # 2 the shared state
    topic: str
    outline: str
    post: str

def make_outline(state: State):                             # 3 node 1
    msg = model.invoke(f"Write a 3-bullet outline about {state['topic']}")
    return {"outline": msg.content}                         # update just the 'outline'
                                                            field

def write_post(state: State):                               # 4 node 2 (sees outline from
node 1)
    msg = model.invoke(f"Expand this outline into a short post:\n{state['outline']}")
    return {"post": msg.content}

builder = StateGraph(State)                                # 5 build
builder.add_node("outline", make_outline)
builder.add_node("write", write_post)
builder.add_edge(START, "outline")                        # 6 wiring
builder.add_edge("outline", "write")
builder.add_edge("write", END)

graph = builder.compile()                                  # 7 turn blueprint into a
runnable

result = graph.invoke({"topic": "why developers love coffee"}) # 8
print(result["post"])

```

Line	Explanation
2	State has three fields. Each node will fill in one of them; all use the default overwrite reducer.
3–4	Nodes are plain functions: take the state, do work, return a partial dict updating only the field(s) they own. Node 2 can read <code>state["outline"]</code> because node 1 wrote it first.
5–6	Register the nodes, then connect them: <code>START → outline → write → END</code> .
7	<code>compile()</code> validates the graph and returns a Runnable (so it has <code>.invoke</code> / <code>.stream</code> , like everything else).
8	Invoke with the initial state (just the topic). The graph runs both nodes and returns the final state with all fields filled.

⚠ USE START/END, NOT SET_ENTRY_POINT

Older tutorials call `builder.set_entry_point("outline")` and `set_finish_point(...)`. These are deprecated in LangGraph v1.0. Use `add_edge(START, ...)` and `add_edge(..., END)` instead.

Swap to local with `from langchain_ollama import ChatOllama; model = ChatOllama(model="llama3.2")` — the graph code is identical.

10.5 — Conditional edges: branching

So far the path is fixed. **Conditional edges** let the graph choose the next node at runtime based on the state — true branching. Here a classifier node decides whether the input is a question or a complaint, then routes to the right handler.

```
graph LR
    START --> classify[classify]
    classify -- "question" --> answer[answer]
    answer --> END
    classify -- "complaint" --> apologize[apologize]
    apologize --> END
```

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    text: str
    category: str
    reply: str

def classify(state: State):
    msg = model.invoke(
        f"Classify as one word, 'question' or 'complaint': {state['text']}"
    )
    return {"category": "complaint" if "complaint" in msg.content.lower() else
"question"}

def answer(state: State):
    return {"reply": model.invoke(f"Answer helpfully: {state['text']}").content}

def apologize(state: State):
    return {"reply": model.invoke(f"Respond with empathy and an apology:
{state['text']}").content}

def route(state: State) -> str:
    # returns the NAME of the next node
    return state["category"]

builder = StateGraph(State)
builder.add_node("classify", classify)
builder.add_node("answer", answer)
builder.add_node("apologize", apologize)

builder.add_edge(START, "classify")
builder.add_conditional_edges("classify", route, {
    "question": "answer",
    "complaint": "apologize",
})
```

```
builder.add_edge("answer", END)
builder.add_edge("apologize", END)

graph = builder.compile()
print(graph.invoke({"text": "My order arrived broken!"})["reply"]) # -> apologize path
```

Piece	Explanation
<code>route(state)</code>	A plain function that inspects the state and returns a string — the key telling the graph where to go next.
<code>add_conditional_edges("classify", route, {...})</code>	After the <code>classify</code> node, run <code>route</code> ; map its returned string to the next node via the dict. <code>"complaint"</code> → <code>apologize</code> , <code>"question"</code> → <code>answer</code> .

10.6 — Loops: the thing chains can't do

This is LangGraph's signature power: an edge can point *back* to an earlier node, creating a loop that repeats until a condition says stop. Here we generate a tagline and keep retrying until it's short enough (or we hit a try limit).

```
START → [generate] → is it good? – no (retry) ↗
                                   |
                                   ↘ back to [generate]
                                   |
                                   ↘ yes / out of tries → END
```

```
from dotenv import load_dotenv
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langchain_google_genai import ChatGoogleGenerativeAI

load_dotenv()
model = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=1.0) # varied tries

class State(TypedDict):
    topic: str
    tagline: str
    attempts: int

def generate(state: State):
    msg = model.invoke(f"Write ONE punchy tagline (max 40 chars) for: {state['topic']}")
    return {
        "tagline": msg.content.strip().strip('"'),
        "attempts": state.get("attempts", 0) + 1, # counter: overwrite reducer
    }

def check(state: State) -> str: # the loop's exit decision
    if len(state["tagline"]) <= 40 or state["attempts"] >= 3:
        return "done"
```

```

    return "retry"

builder = StateGraph(State)
builder.add_node("generate", generate)
builder.add_edge(START, "generate")
builder.add_conditional_edges("generate", check, {          # loop or exit
    "retry": "generate",  # ← edge points BACK = a loop
    "done": END,
})
graph = builder.compile()

result = graph.invoke({"topic": "a coffee shop for programmers", "attempts": 0})
print(result["tagline"], f"(after {result['attempts']} tries)")

```

Piece	Explanation
<code>attempts</code>	A counter in state, incremented each pass. The overwrite reducer is right here — we want the latest count, not a history.
<code>check</code>	Returns <code>"done"</code> if the tagline is short enough <i>or</i> we've tried 3 times; else <code>"retry"</code> . The try-limit guarantees the loop ends.
<code>"retry": "generate"</code>	The loop. The conditional edge routes back to <code>generate</code> , which runs again with the incremented counter.

ALWAYS GIVE A LOOP AN EXIT

A graph that loops with no stopping condition runs forever (LangGraph has a recursion limit that will eventually error, but don't rely on it). Every loop needs a guaranteed exit — here, the `attempts >= 3` cap. Think about your stop condition *before* you wire the back-edge.

10.7 — Streaming & inspecting a graph

To watch a graph run step by step (great for debugging loops and branches), use `.stream()` with a `stream_mode`:

```

for step in graph.stream({"topic": "a coffee shop for programmers", "attempts": 0},
                        stream_mode="updates"):  # yields each node's update as it runs
    print(step)

```

stream_mode	Yields
<code>"updates"</code>	Just what each node changed this step — best for seeing the flow.
<code>"values"</code>	The full state after each step.

And to see the graph's shape (nodes, edges, branches), reuse the Phase 4 trick:

```
graph.get_graph().print_ascii() # draws the node/edge diagram
```

10.8 — When to use a chain vs an agent vs a graph

Use...	When	From
LCEL chain ()	A fixed, linear pipeline: prompt → model → parser, RAG Q&A, classification. No loops or branching needed.	Phases 1–8
create_agent	You want a model to use tools in a loop and you're happy with the standard agent behaviour.	Phase 9
Hand-built StateGraph	You need custom control flow: specific branches, loops, multiple cooperating steps, human-in-the-loop, or a shape the prebuilt agent doesn't fit.	This phase

DON'T OVER-REACH FOR GRAPHS

Most tasks are happiest as a chain or a prebuilt agent — reach for a hand-built graph only when you genuinely need the extra control. A graph is more code and more to maintain. The skill is knowing it's *there* when a chain can't express your logic, not using it for everything.

10.9 — Common errors & troubleshooting

Symptom	Likely cause & fix
Graph not compiled / no .invoke	You forgot graph = builder.compile() . Build the blueprint, then compile to get a runnable.
Node update seems ignored / state didn't change	A node must return a dict of updates. Returning None or printing without returning leaves state unchanged.
A field keeps overwriting when you wanted to accumulate	Add a reducer: Annotated[list, add] (or add_messages for messages).
set_entry_point deprecation/ errors	Use add_edge(START, ...) / add_edge(..., END) (§10.4).
Loop never ends / recursion-limit error	No guaranteed exit. Add a counter and an exit condition (§10.6).
Conditional edge always goes one way	Your route function returns a string that doesn't match the mapping keys. Print its return value; ensure keys line up exactly.
KeyError reading a state field in a node	The field wasn't set yet (no earlier node wrote it, or it wasn't in the initial input). Provide it in the initial state or via state.get(key, default) .

10.10 — Mini practice exercise

YOUR TURN (DO THIS BEFORE PHASE 11)

Task: Build a "draft → critique → revise" graph. State holds `topic`, `draft`, `critique`, and `revisions` (a count). Node 1 writes a short draft. Node 2 critiques it. A conditional edge checks the critique: if it contains the word "good" (or you've revised twice), go to END; otherwise route to a **revise** node that rewrites the draft using the critique — then loop back to the critique node. Stream the run with `stream_mode="updates"` so you can watch each pass. Try it on Gemini and Ollama.

Why: the self-improvement loop (generate → judge → improve) is one of the most useful agentic patterns in practice, and it's impossible with a plain chain. Building it by hand cements nodes, conditional edges, loops, and reducers all at once.

Hints (not the full answer):

- Keep a `revisions` counter and cap it (e.g. `>= 2`) so the loop always exits.
- The critique node should output text the router can test (ask the model to start with GOOD or REVISE).
- The back-edge goes `revise → critique` (re-judge the new draft), not `revise → draft`.
- Use `print_ascii()` to confirm your wiring looks like the loop you intended.

Terms introduced this phase

Term	One-line meaning
LangGraph	Library for workflows as graphs of nodes + edges with shared state; supports loops/branches.
State	A <code>TypedDict</code> of fields passed to every node during a run.
Node	A function: reads state, returns a partial dict of updates.
Edge	A route between nodes; <code>add_edge</code> for fixed transitions.
Conditional edge	A runtime branch: a router function returns the next node's name.
Reducer	How a returned field merges into state: overwrite (default) or append (<code>add</code> / <code>add_messages</code>).
START / END	Special markers for the graph's entry and exit (replace <code>set_entry_point</code>).
compile()	Validates the graph and returns a runnable; add <code>checkpointers=</code> for memory.
Loop / cycle	An edge back to an earlier node; must have a guaranteed exit condition.
stream_mode	<code>"updates"</code> (per-node changes) or <code>"values"</code> (full state) when streaming a graph.

PHASE 10 COMPLETE

You can design a typed State with the right reducers, build nodes and wire them with fixed and conditional edges, create loops that repeat until done, and stream/inspect the whole thing — the machinery behind every agent you'll build. **Next — Phase 11: Observability & Deployment**, where we make your apps

production-ready: trace every step with **LangSmith** to see exactly what's happening, then wrap a chain or agent in a simple web API so others can call it.